

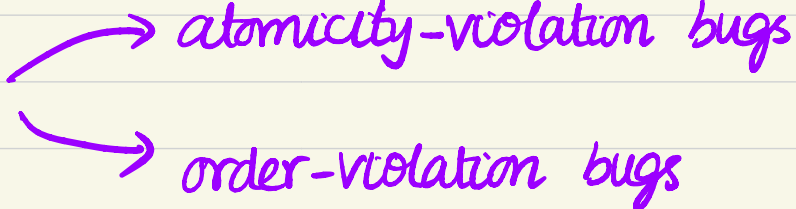
Common concurrency problems

Question: what types of concurrent bugs manifest in complex, concurrent programs?

How to handle such bugs?

Application	What it does	Non-Deadlock	Deadlock
MySQL	Database Server	14	9
Apache	Web Server	13	4
Mozilla	Web Browser	41	16
OpenOffice	Office Suite	6	2
Total		74	31

Study by Lu et al. : Bugs In Modern Applications

Non-deadlock bugs 

- atomicity-violation bugs
- order-violation bugs

Atomicity-violation bugs

an example
found in SQL

```
1 Thread 1::
2 if (thd->proc_info) {
3     fputs(thd->proc_info, ...);
4 }
5
6 Thread 2::
7 thd->proc_info = NULL;
```

what is the bug in the above code ?

- Thread 1 reads `thd->proc_info` as **non-NULL** and enters the **if** block
- Before it calls `fputs()`, thread 2 enters and sets it to **NULL**.
- Thread 1 then dereferences `thd->proc_info`, which is now **NULL**, leading to **undefined behaviour**.

Problem: a code region is intended to be atomic, but the atomicity is not enforced during execution.

How to fix the above bug?

```
1 pthread_mutex_t proc_info_lock = PTHREAD_MUTEX_INITIALIZER;
2
3 Thread 1::
4 pthread_mutex_lock(&proc_info_lock);
5 if (thd->proc_info) {
6     fputs(thd->proc_info, ...);
7 }
8 pthread_mutex_unlock(&proc_info_lock);
9
10 Thread 2::
11 pthread_mutex_lock(&proc_info_lock);
12 thd->proc_info = NULL;
13 pthread_mutex_unlock(&proc_info_lock);
```

ensure that when either thread access `proc_info`, field, it has a lock held (`proc_info_lock`)

Order-violation bugs

```
1 Thread 1::  
2 void init() {  
3     mThread = PR_CreateThread(mMain, ...);  
4 }  
5  
6 Thread 2::  
7 void mMain(...) {  
8     mState = mThread->State;  
9 }
```

What is the bug here?

A possible sequence of execution is as follows:

- Thread 1 calls `PR_CreateThread()`
- The runtime creates a new thread and starts running `mMain()`.
- Before Thread 1 stores the return value of `PR_CreateThread()` into `mThread`, Thread 2 executes `mThread->State`
- Thread 2 dereferences an uninitialized or garbage pointer → undefined behaviour.

Problem: The desired order between two (groups of) memory access could get flipped as the order is not enforced during execution.

How to fix this bug? → Using condition variables, the code can be corrected as follows.

```

1 pthread_mutex_t mtLock = PTHREAD_MUTEX_INITIALIZER;
2 pthread_cond_t  mtCond = PTHREAD_COND_INITIALIZER;
3 int mtInit      = 0;
4
5 Thread 1::
6 void init() {
7     ...
8     mThread = PR_CreateThread(mMain, ...);
9
10    // signal that the thread has been created...
11    pthread_mutex_lock(&mtLock);
12    mtInit = 1;
13    pthread_cond_signal(&mtCond);
14    pthread_mutex_unlock(&mtLock);
15    ...
16 }
17
18 Thread 2::
19 void mMain(...) {
20     ...
21     // wait for the thread to be initialized...
22     pthread_mutex_lock(&mtLock);
23     while (mtInit == 0)
24         pthread_cond_wait(&mtCond, &mtLock);
25     pthread_mutex_unlock(&mtLock);
26
27     mState = mThread->State;
28     ...
29 }

```

Remark: Most often non-deadlock bugs can be likely fixed as in the examples above.

A classic problem that arises in concurrent systems with complex protocols.

↳ deadlock

eg:- Thread 1 is holding a lock L1 and waiting for another one L2
and Thread 2 " " L2 " " L1

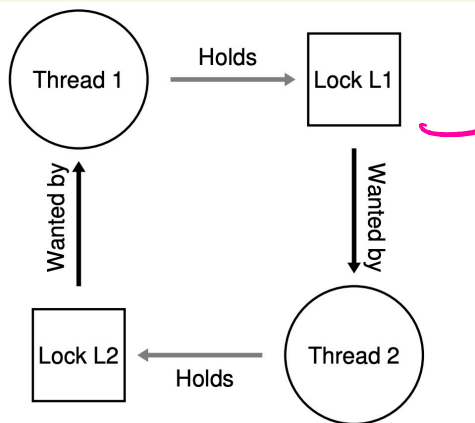
Thread 1:

① pthread_mutex_lock (L1);
③ pthread_mutex_lock (L2);

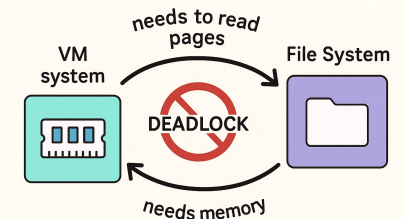
Thread 2:

② pthread_mutex_lock (L2);
④ pthread_mutex_lock (L1);

a deadlock happens when the code gets executed in the above order.



the deadlock dependency graph
(a cycle in the graph is indicative of the deadlock)



Why do deadlocks happen? might be due to complex dependencies

b/w components or encapsulation

eg:- dependencies b/w the virtual memory system and file system.

Necessary conditions for deadlock

1. Mutual exclusion: threads claim exclusive control of resources that they acquire ^{like, locks}
2. Hold-and-wait: threads hold resources allocated to them while waiting for additional resources.
3. No Preemption: Resources cannot be forcibly removed from threads that are holding them.
4. Circular wait: There exists a circular chain of threads such that each thread holds one or more resources that are being requested by the next thread in chain.

If any of the above 4 conditions are not met, deadlock cannot occur.

What is a strategy to prevent deadlock?

→ prevent one of the above conditions.

Prevention: circular wait

↳ provide a total ordering on lock acquisition.

eg:- if there are two locks L_1 and L_2 → say, we always acquire L_1 before L_2 .

strict ordering $\Rightarrow$ no cyclical wait $\Rightarrow$ no deadlock!

problem: a total lock ordering may be difficult to achieve.

$\hookrightarrow$ can be replaced even with a partial ordering
eg:- memory mapping code in Linux.

Prevention: Hold-and-wait

$\hookrightarrow$ acquire all locks at once, atomically.

```
1 pthread_mutex_lock(prevention); // begin acquisition
2 pthread_mutex_lock(L1);
3 pthread_mutex_lock(L2);
4 ...
5 pthread_mutex_unlock(prevention); // end
```

$\hookrightarrow$ ensures that no untimely thread switch can occur in the midst of lock acquisition $\rightarrow$ no hold-and-wait situation
 $\Rightarrow$ no deadlock.

Problem: - need to know exactly which locks must be held and acquire them ahead of time $\rightarrow$ cause lack of encapsulation.
- decrease concurrency.

Dealing with No Preemption

Problem with multiple lock acquisition

→ When waiting for one lock, we are holding another.

a flexible way to do this → routine `pthread_mutex_trylock()`

either grabs the lock (if available)
and return success

returns an error
indicating the lock is held.

try again later if
you want to grab that lock.

```
1 top:
2   pthread_mutex_lock(L1);
3   if (pthread_mutex_trylock(L2) != 0) {
4       pthread_mutex_unlock(L1);
5       goto top;
6   }
```

Here, although no external force is taking the lock away, the design effectively allows preemption of a resource.

↳ a thread gives up its currently held lock to prevent a deadlock from forming.

But consider the following scenario.

→ Two threads use the same "try-release-retry" logic but they acquire locks in opposite order.

e.g., suppose

① thread A locks L1

② thread B locks L2

③ both then try to acquire the other lock with

`pthread_mutex_trylock()` and both fail

④ each release its own lock and goes back to the top

⑤ they both retry - again roughly the same time - so the pattern repeats.

they never block, so the system isn't deadlocked (each thread keeps running)
but,

One new problem: livelock

↳ neither thread makes progress toward its goal.

How to deal livelock? → add a random delay before looping back and retrying → decrease the chance of repeated interference.

Remark: the retrying routine need to be used cautiously! (think, why?)

Dealing mutual exclusion

How to avoid the need for mutual exclusion?

↳ lock-free approaches

↳ use powerful hardware instructions to build data structures s.t. no explicit locking is required.

Recall **compareAndSwap** instruction provided by the hardware that does the following atomically.

```
1 int CompareAndSwap(int *address, int expected, int new) {  
2     if (*address == expected) {  
3         *address = new;  
4         return 1; // success  
5     }  
6     return 0; // failure  
7 }
```

How to atomically increment a value by a certain amount using **compareAndSwap**?

```

1 void AtomicIncrement(int *value, int amount) {
2     do {
3         int old = *value;
4     } while (CompareAndSwap(value, old, old + amount) == 0);
5 }

```

no lock is acquired $\Rightarrow$ no deadlock!

Exercise: Verify the above approach increment a value atomically!

Exercise: Explore more such examples, like, list insertion using
using compare And swap.

A final general strategy to deal deadlock

Allow deadlocks to occur occasionally

$\rightarrow$ take action when it is detected.

restart or requires a human
intervention

$\hookrightarrow$ by periodically
running a deadlock detector
 $\rightarrow$ build a resource graph
 $\rightarrow$ check for cycles.